SIC and SIC/XE Assembler and Loader College AAST

Systems Programming college project documentation and technical report

This project demonstrates how assembly source becomes object records and how a loader places object code into a memory map. It contains two related Python programs: a SIC assembler and a SIC/SIC-XE loader. The supplied loader examples are separate sample inputs; the repository does not wire the assembler output directly into the loader.

What the project teaches

- Why assembly needs two passes: addresses and symbols must be known before object code can be generated.
- How H, T and E records describe a program for loading.
- How absolute loading differs from relocation and external-symbol linking.
- How a memory map represents bytes at hexadecimal addresses.

The two paths

Path	Input	Main result
Assembler	in.txt (SIC source)	intermediate, pass outputs, symbol table, HTE object file
Absolute loader	sic.txt (dot-separated object records)	memory grid at stated object addresses
Linking loader	sicxe.txt (caret-separated records)	external symbol table and relocated memory grid

Status: This is a code-based explanation. Syntax compilation passed, but execution was not verified because pandas, numpy and pandasgui are unavailable in this environment.

1. Architecture scheme

The assembler and loader share the idea of object records, but use different included samples and different separators. The scheme below shows the actual repository paths.

SIC source / in.txt
Clean input / Intermediate.py
Address + symbols / Pass1.py
Object bytes / Pass2.py
H/T/E output / HTE.py

The loader is started separately. Choose one route in loader.py. Either route ends in a memory table displayed with pandasgui.

SIC route	SIC/XE route
sic.txt -> absolute()	sicxe.txt -> extable()

Why the stages exist

Stage	Purpose
Input cleanup	Remove comment rows and preserve label, operation and operand fields.
Pass 1	Advance the location counter and record label addresses.
Pass 2	Resolve labels and produce instruction or data bytes.
Object records	Package bytes with program metadata and execution start.
Loader	Place bytes in memory; the SIC/XE path also uses external definitions and modifications.

The file main.py in the loader folder is commented-out draft code. run.bat starts loader.py, which contains the active menu.

2. Assembler workflow

Read in.txt
Drop comments
Pass 1: LC + symbols
Pass 2: bytes
Write H/T/E

Intermediate.py runs Cleaninp('in.txt') when imported or executed, then its main function calls pass1, pass2 and hte in order. Files are read and written relative to the current working directory, so the script should be launched from its own folder.

What each assembler file does

File	Role
in.txt	Sample SIC COPY source, including line numbers and comments.
Intermediate.py	Drops first and last input columns and comment lines; writes intermediate.txt; orchestrates passes.
Pass1.py	Calculates location counter values for instructions and BYTE, WORD, RESB, RESW; writes out_pass1.txt and symbTable.txt.
Pass2.py	Maps opcodes and operands to object bytes; writes out_pass2.txt.
HTE.py	Builds header, text and end records in hte.txt.
SymbolTable.py	A one-line pandas import; it does not implement the symbol table logic.

Record meaning

- H: program name, start address and program length.
- T: a start address, byte count and a contiguous sequence of object bytes.
- E: the execution entry address.

Example: the source begins COPY START 1000. The saved symbol table places FIRST at 1000 and CLOOP at 1003. The saved hte.txt instead begins with HFIRST 001003..., which is an observed inconsistency.

3. Loader workflow

Run loader.py
Select 1 or 2
Parse object records
Build memory grid
Display grid

The active loader menu accepts 1 for the SIC absolute loader or 2 for the SIC/XE linking loader. Its functions use pandas DataFrames as a byte-addressed memory grid: each row represents 16 addresses and columns 0 through F hold one byte each. Unfilled positions are shown as x.

SIC absolute loading

absolute('sic.txt') reads dot-separated H, T and E records. It takes the header start and length, creates memory rows, splits each T record's object string into byte pairs, and writes those pairs at the record's stated addresses. It then calls pandasgui.show to display the grid.

SIC/XE linking loading

extable('sicxe.txt') reads caret-separated control sections. H records identify sections and lengths; D records define external symbols; R records list referenced names; T records contain bytes; M records request address changes; E ends a section. The function writes Ext_sym_Table.txt, places text bytes, applies its modification logic and displays the grid.

Record	Meaning in the SIC/XE sample
H	Control section name, origin and length.
D	External symbols defined by that section.
R	External symbols referenced by that section.
T	Object bytes and their relative start.
M	Field to modify, field length, sign and symbol.
E	End and optional entry point.

The linking path currently fixes its initial load address at hexadecimal 403F, and parts of the code assume the layout of the included sample.

4. Example and file map

Assembler example

The sample program begins at address 1000. FIRST STL RETADR is at 1000 and CLOOP JSUB RDREC is at 1003 in the saved pass output. The saved symbol table assigns RDREC to 2039. Pass 2 combines opcodes and addresses into object code, such as 141033 for STL RETADR and 482039 for JSUB RDREC.

Loader examples

sic.txt contains a single object program named COPY and its H/T/E records. sicxe.txt contains three control sections (XXPROG1, XXPROG2 and XXPROG3), external definitions and modification records. These are demonstration inputs, not a direct export from the included assembler.

Folder or artifact	Use
projeytc syetsm/projeytc syetsm	Assembler source, input, generated text files and historical PDFs.
loader/loader/loader	Loader source, batch launcher, sample object files and historical Word report.
out_pass1.txt / symbTable.txt	Addressed listing and label lookup.
out_pass2.txt / hte.txt	Object-code listing and H/T/E output.
Ext_sym_Table.txt	External symbol table written by the linking loader.
*.rar and __pycache__	Archives and Python bytecode; not required for understanding the source flow.

How to run from source

- Install Python with pandas, numpy and pandasgui in a compatible environment.
- From the assembler's inner folder, run: python Intermediate.py.
- From the loader's inner folder, run: python loader.py (or run.bat), then choose 1 or 2.

Running these commands writes or rewrites output text files in the project folders. Review or copy the samples first if their current contents matter.

5. Implementation notes and next steps

The code is useful as a course demonstration, but the saved outputs and several branches need validation before it can be presented as a complete general-purpose assembler and loader.

Finding	Effect / recommended check
HTE start mismatch	HTE.py reads df.iloc[1,0] as the start and df.iloc[0,1] as the name. Compare generated header and E address with START and END directives.
Pending text not flushed	HTE.py writes a T record only when a reservation or size threshold is reached. Check that the final nonempty byte run is written before E.
Sample-specific linking logic	extable() hardcodes 403F and uses record-position assumptions. Parameterize the load address and test each control section independently.
Limited validation	Unknown instructions, undefined or duplicate labels, malformed records and bounds are not handled systematically. Add clear errors and fixtures.
Environment dependency	The loader opens pandasgui; it needs a desktop GUI environment. Package or document compatible dependency versions.

Suggested verification sequence

- Assemble the sample and compare each pass output with its source line and location counter.
- Parse the generated H/T/E records and confirm each T length equals its byte count.
- Load known SIC bytes and verify addresses and row boundaries in the memory grid.
- Test SIC/XE modifications for positive and negative references, field lengths and multiple control sections.

Glossary

LC: location counter. Opcode: numeric operation code. Symbol table: label-to-address map. Object code: encoded bytes. Control section: separately loadable program part. Relocation: changing address fields for a chosen load address. Linking: resolving references between control sections.

Source basis: the Python source and sample text files in this project.